

COMP 421: Files & Databases

Lecture 8: Hash Tables

Announcements

Project #1: less than 80%, come to my office hours or make an appointment. Must be working to complete project 2.

Project #2 released

Code review 1 sign ups posted

Project #2

You will build a thread-safe B+tree backed by your buffer pool manager.

- Page Layout
- Insert/Delete/Find Operations
- Iterator
- Latch Crabbing

We define the API for you. You need to provide the method implementations.

WARNING:

**This is more difficult than Project #1.
Start immediately!**

Task #1: Page Layouts

- How each node will store its key/values in a page.
- You only need to support unique keys.

Task #2: Operations

- Support point queries (single key).
- Support inserts with node splitting.
- Support removal of keys with sibling stealing + merging.
- Does not need to be thread-safe.

Task #3: Index Iterator

- Create a STL iterator for range scans on leaf nodes.
- You only need to support ascending scans.

Task #4: Concurrent Index

- Introduce latch crabbing/coupling protocol to support safe concurrent operations.
- Make sure you have splits / merges working correctly before proceeding with this task.

Latch Crabbing/Coupling

Protocol to allow multiple threads to access/modify B+Tree at the same time.

- Get latch for parent
- Get latch for child
- Release latch for parent if “safe”

A **safe node** is one that will not split or merge when updated.

- Not full (on insertion)
- More than half-full (on deletion)

Latch Crabbing/Coupling

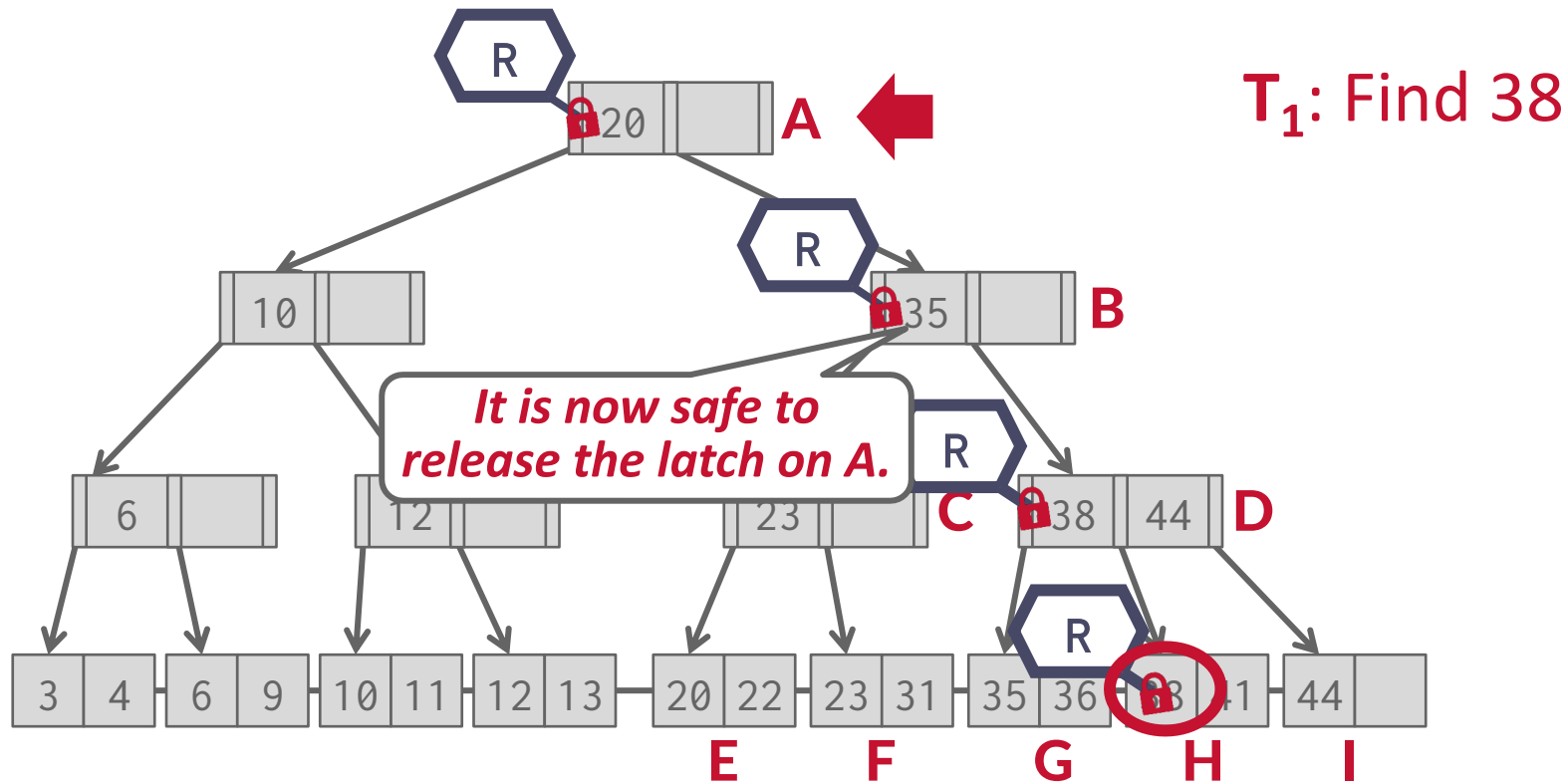
Find: Start at root and traverse down the tree:

- Acquire **R** latch on child,
- Then unlatch parent.
- Repeat until we reach the leaf node.

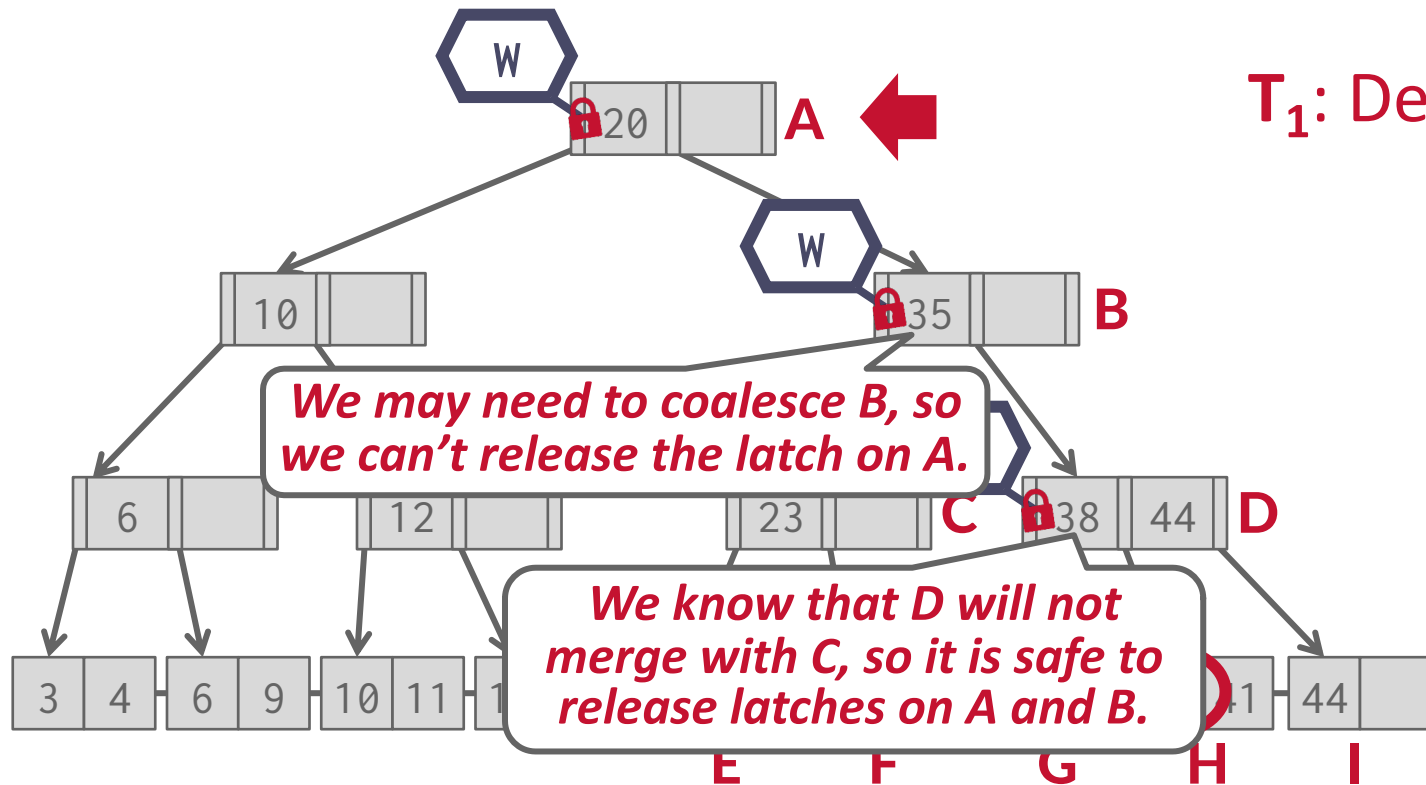
Insert/Delete: Start at root and go down, obtaining **W** latches as needed. Once child is latched, check if it is safe:

- If child is safe, release all latches on ancestors

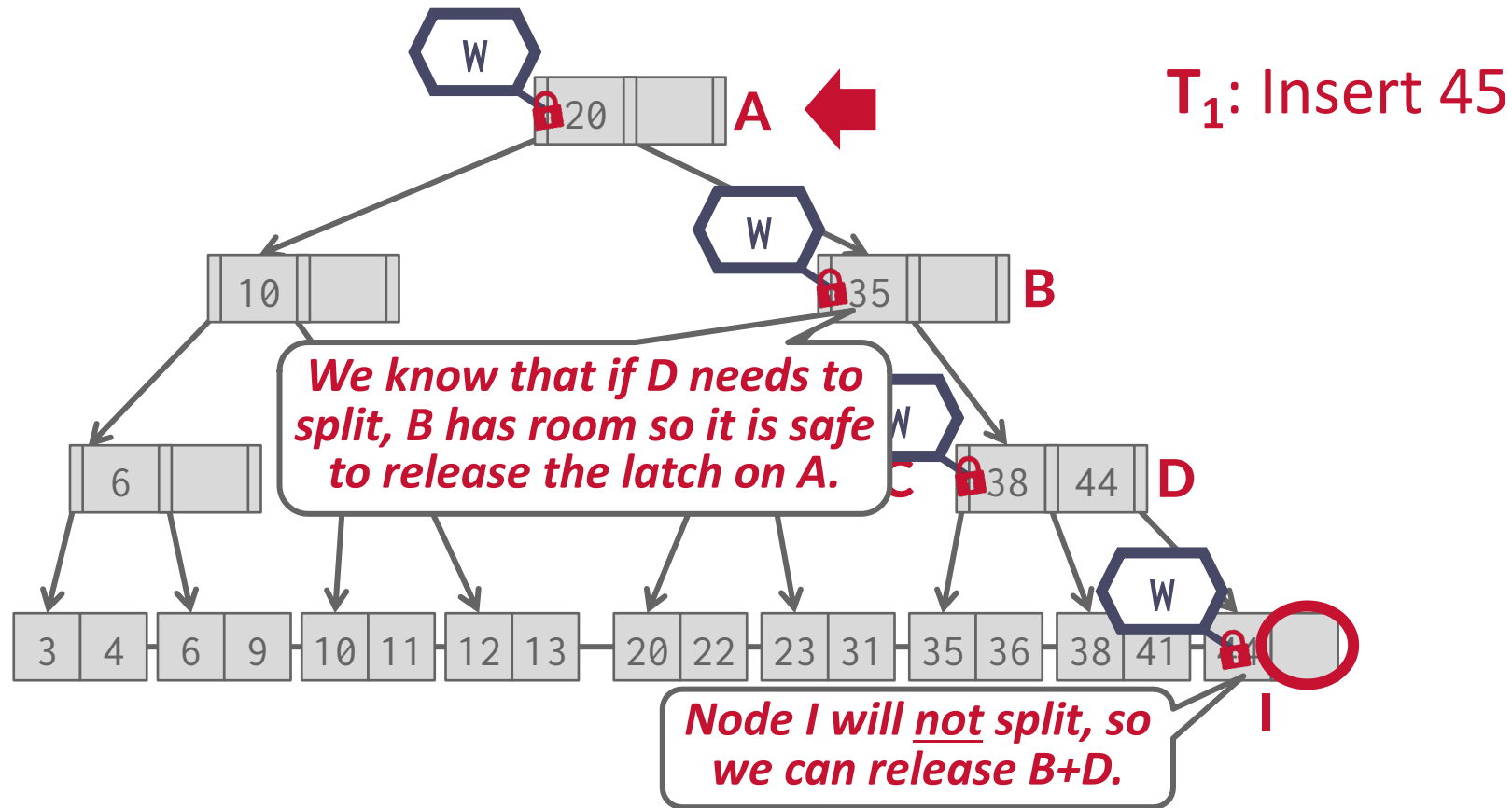
Example #1 – FIND 38



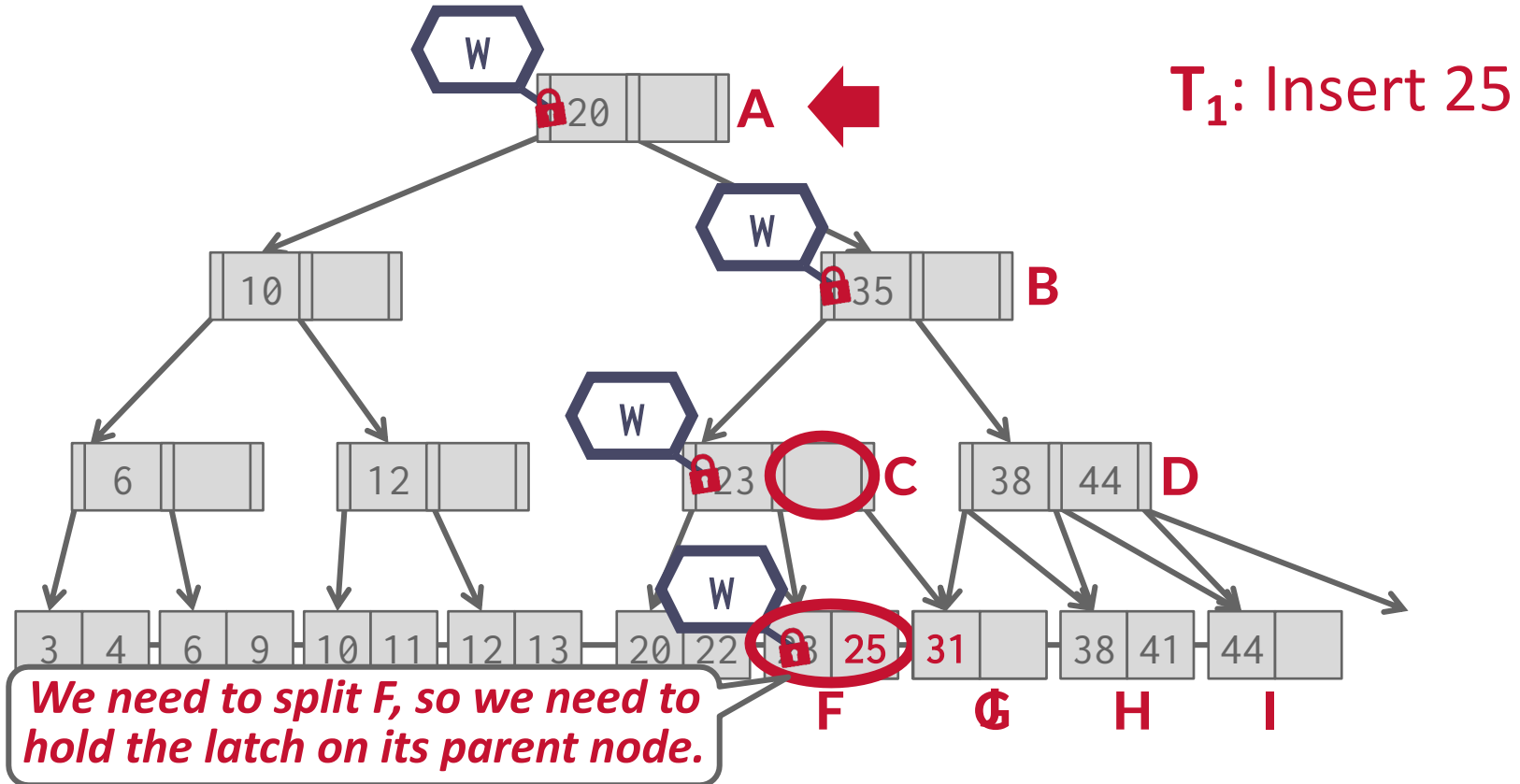
Example #2 – DELETE 38



Example #3 – INSERT 45



Example #4 – INSERT 25



Development Hints

Follow the textbook semantics and algorithms.

Set the page size to be small (e.g., 512B) when you first start so that you can see more splits/merges.

Make sure that you protect the internal B+Tree **root_page_id** member.

Extra Credit

Gradescope Leaderboard runs your code with a specialized in-memory version of BusTub.

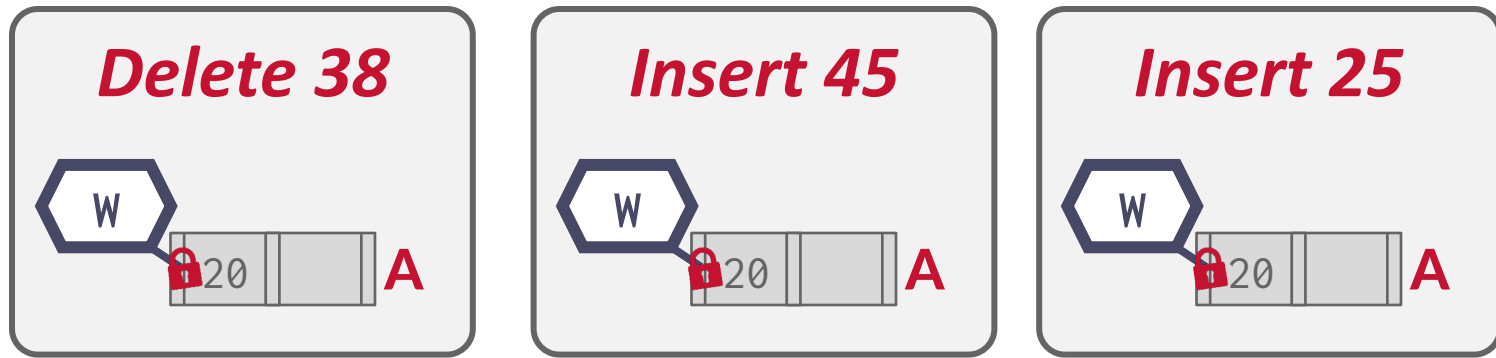
The top 20 fastest implementations in the class will receive extra credit for this assignment.

- **#1**: 50% bonus points
- **#2–5**: 25% bonus points
- **#6–10**: 10% bonus points

You must pass all the test cases to qualify!

Observation

What was the first step that all the update examples did on the B+Tree?



Taking a write latch on the root every time becomes a bottleneck with higher concurrency.

Better Latching Algorithm

Most modifications to a B+Tree will not require a split or merge.

Instead of assuming there will be a split/merge, optimistically traverse the tree using read latches.

If a worker guesses wrong, repeat traversal with pessimistic algorithm.

Acta Informatica 9, 1–21 (1977)



Concurrency of Operations on B-Trees

R. Bayer* and M. Schkolnick
IBM Research Laboratory, San José, CA 95193, USA

Summary. Concurrent operations on B-trees pose the problem of insuring that each operation can be carried out without interfering with other operations being performed simultaneously by other users. This problem can become critical if these structures are being used to support access paths, like indexes, to data base systems. In this case, serializing access to one of these indexes can create an unacceptable bottleneck for the entire system. Thus, there is a need for locking protocols that can assure integrity for each access while at the same time providing a maximum possible degree of concurrency. Another feature required from these protocols is that they be deadlock free, since the cost to resolve a deadlock may be high.

Recently, there has been some questioning on whether B-tree structures can support concurrent operations. In this paper, we examine the problem of concurrent access to B-trees. We present a deadlock free solution which can be tuned to specific requirements. An analysis is presented which allows the selection of parameters so as to satisfy these requirements.

The solution presented here uses simple locking protocols. Thus, we conclude that B-trees can be used advantageously in a multi-user environment.

1. Introduction

In this paper, we examine the problem of concurrent access to indexes which are maintained as B-trees. This type of organization was introduced by Bayer and McCreight [2] and some variants of it appear in Knuth [10] and Wedekind [13]. Performance studies of it were restricted to the single user environment. Recently, these structures have been examined for possible use in a multi-user (concurrent) environment. Some initial studies have been made about the feasibility of their use in this type of situation [1, 6], and [11].

An accessing schema which achieves a high degree of concurrency in using the index will be presented. The schema allows dynamic tuning to adapt its performance to the profile of the current set of users. Another property of the

* Permanent address: Institut für Informatik der Technischen Universität München, Arcisstr. 21, D-8000 München 2, Germany (Fed. Rep.)

Better Latching Algorithm

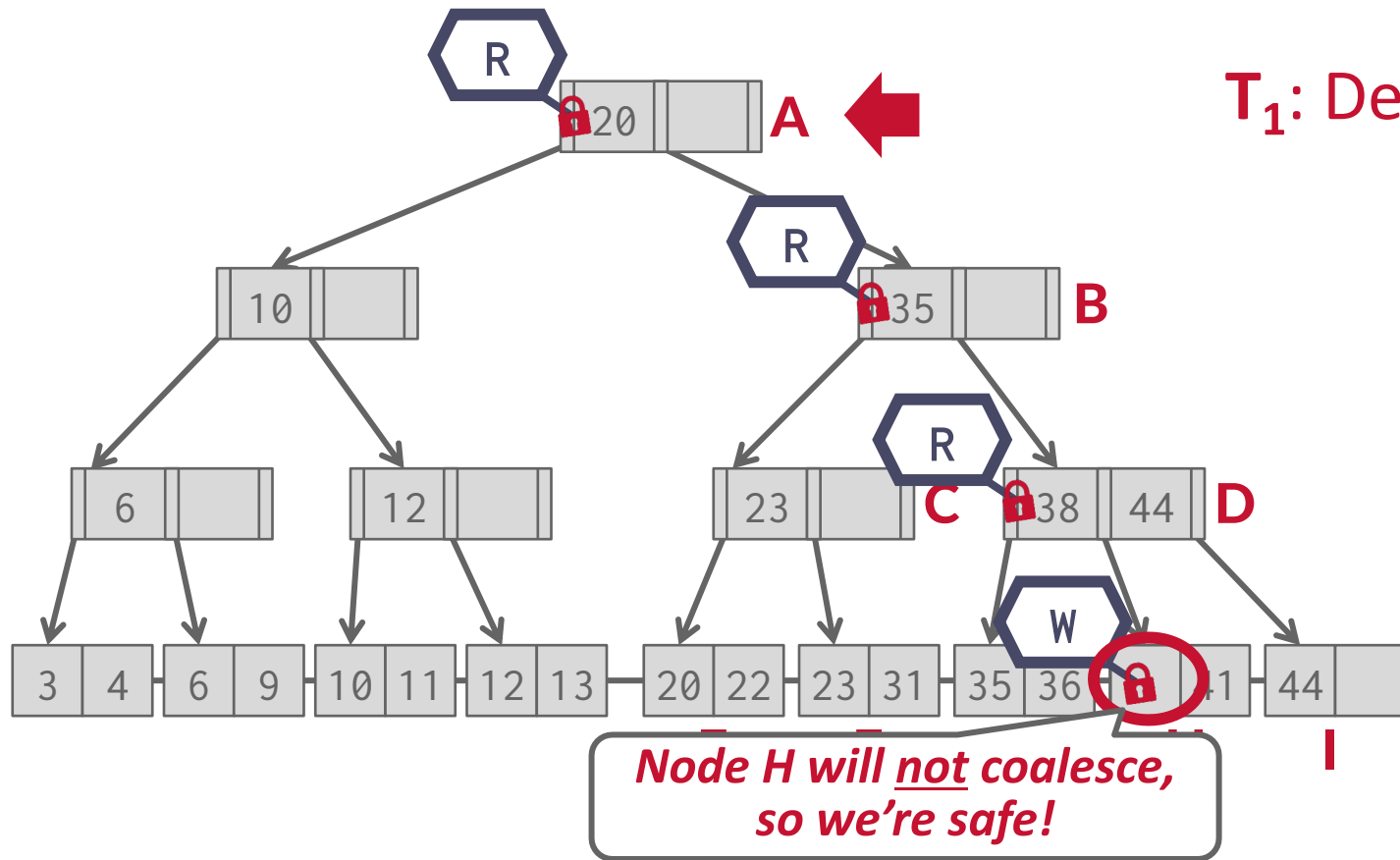
Search: Same as before.

Insert/Delete:

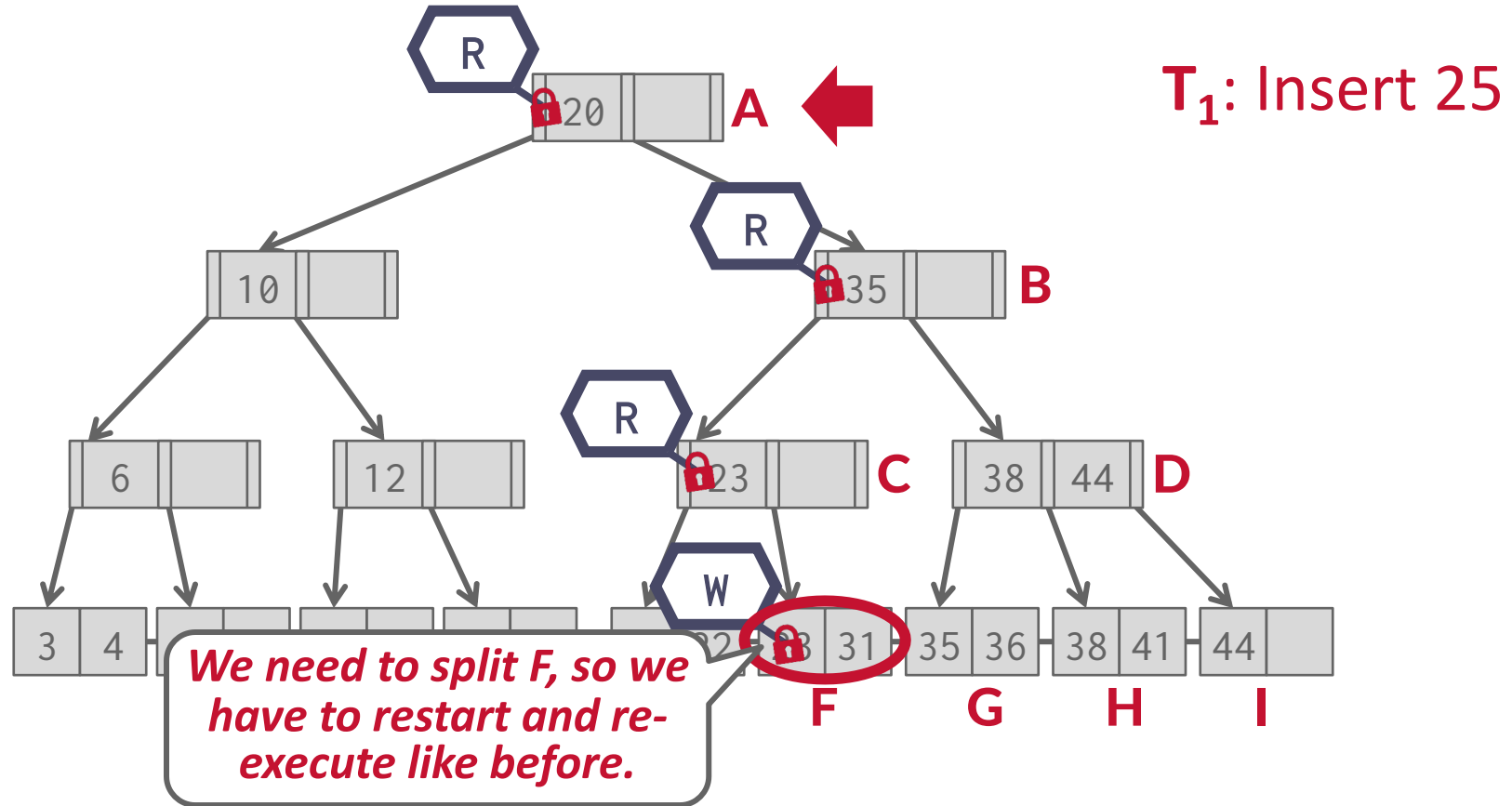
- Set latches as if for search, get to leaf, and set **W** latch on leaf.
- If leaf is not safe, release all latches, and restart thread using previous insert/delete protocol with write latches.

This approach optimistically assumes that only leaf node will be modified; if not, **R** latches set on the first pass to leaf are wasteful.

Example #2 – DELETE 38



Example #4 – INSERT 25



Course Outline

We are now going to talk about how to support the DBMS's execution engine to read/write data from pages.

Two types of data structures:

- Hash Tables (Unordered)
- Trees (Ordered)

Query Planning

Operator Execution

Access Methods

Buffer Pool Manager

Disk Manager

Today's Agenda

Background

Hash Functions

Static Hashing Schemes

Dynamic Hashing Schemes

Data Structures

Internal Meta-data

Core Data Storage

Temporary Data Structures

Table Indexes

Design Decisions

Data Organization

→ How we layout data structure in memory/pages and what information to store to support efficient access.

Concurrency

→ How to enable multiple threads to access the data structure at the same time without causing problems.

Hash Tables

A hash table implements an unordered associative array that maps keys to values.

It uses a hash function to compute an offset into this array for a given key, from which the desired value can be found.

Space Complexity: $O(n)$

Time Complexity:

→ Average: $O(1)$ ← *Databases care about constants!*

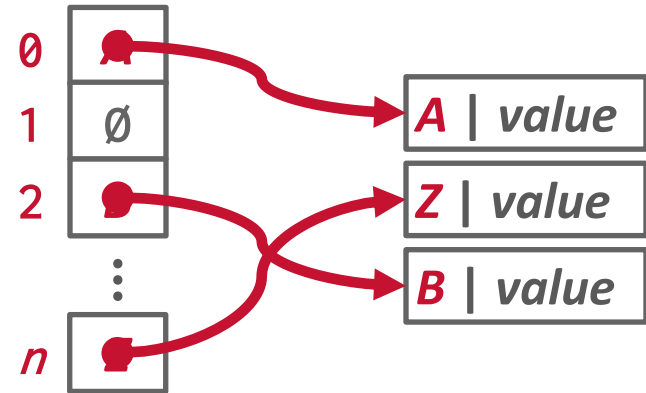
→ Worst: $O(n)$

Static Hash Table

Allocate a giant array that has one slot for every element you need to store.

To find an entry, mod the key by the number of elements to find the offset in the array.

$$\text{hash}(\text{key}) \% n$$



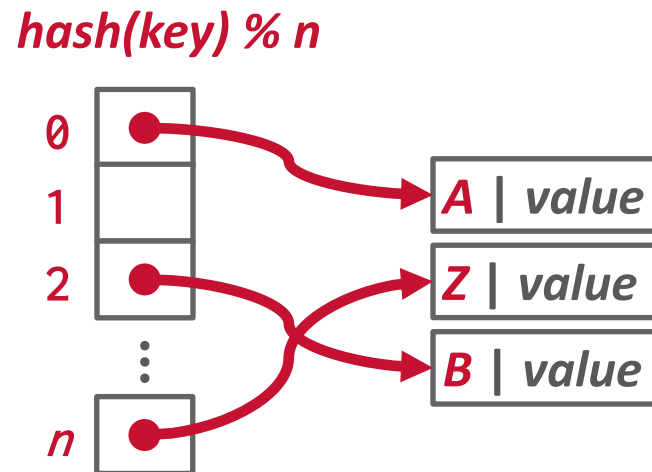
Unrealistic Assumptions

Assumption #1: Number of elements is known ahead of time and fixed.

Assumption #2: Each key is unique.

Assumption #3: Perfect hash function guarantees no collisions.

→ If $\text{key1} \neq \text{key2}$, then
 $\text{hash}(\text{key1}) \neq \text{hash}(\text{key2})$



Hash Table

Design Decision #1: Hash Function

- How to map a large key space into a smaller domain.
- Trade-off between being fast vs. collision rate.

Design Decision #2: Hashing Scheme

- How to handle key collisions after hashing.
- Trade-off between allocating a large hash table vs. additional instructions to get/put keys.

Hash Functions

For any input key, return an integer representation of that key.

→ Converts arbitrary byte array into a fixed-length code.

We want something that is fast and has a low collision rate.

We do not want to use a cryptographic hash function for DBMS hash tables (e.g., SHA-2).

Hash Functions

smhasher

SMhasher

Linux Build status  

Hash function	MiB/sec	cycl./hash	cycl./map	size
donothing32	11149460.06	4.00	-	13
donothing64	11787676.42	4.00	-	13
donothing128	11745060.76	4.06	-	13
NOP_OAAT_read64	11372846.37	14.00	-	41
BadHash	769.94	73.97	-	4
sumhash	10699.57	29.53	-	36
sumhash32	42877.79	23.12	-	86
multiply_shift	8026.77	26.05	226.80 (8)	3
pair_multiply_shift	3716.95	40.22	186.34 (3)	6
crc32	383.12	134.21	257.50 (11)	2
md5_32	350.53	644.31	894.12 (10)	4

Summary

I added some SSE assisted hashes and fast intel/arm CRC32-C, AES and SHA HW variants. See also the old <https://github.com/aappleby/smhasher/wiki>, the improved, but unmaintained fork <https://github.com/demerphq/smhasher>, and the new improved version SMHasher3 <https://gitlab.com/fwojck/smhasher3>.

So the fastest hash functions on x86_64 without quality problems are:

- rapidhash (an improved wyhash)
- xxh3low
- wyhash
- umash (even universal!)
- ahash64
- t1ha2_atonce
- komihash
- FarmHash (*not portable, too machine specific: 64 vs 32bit, old gcc, ...*)
- halftime_hash128
- Spooky32
- pengyhash
- nmhash32
- mx3
- MUM/mir (*different results on 32/64-bit archs, lots of bad seeds to filter out*)
- fasthash32



Static Hashing Schemes

Approach #1: Linear Probe Hashing

Approach #2: Cuckoo Hashing

← **Open Addressing**

More complex schemes exist, rarely used in practice

- Robin Hood Hashing
- Hopscotch Hashing
- Swiss Tables

Linear Probe Hashing

Single giant table of fixed-length slots.

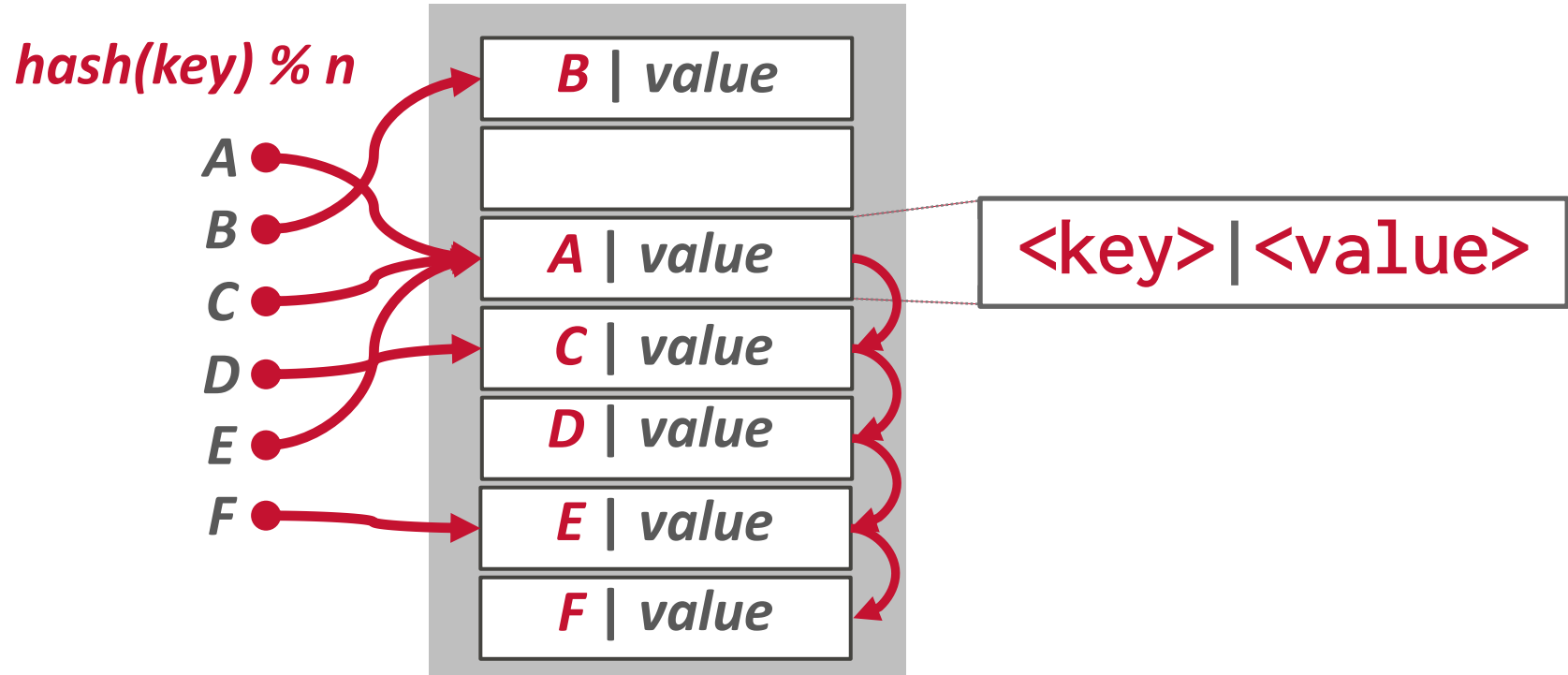
Resolve collisions by linearly searching for the next free slot in the table.

- To determine whether an element is present, hash to a location in the table and scan for it.
- Store keys in table to know when to stop scanning.
- Insertions and deletions are generalizations of lookups.

The table's **load factor** determines when it is becoming too full and should be resized.

- Allocate a new table twice as large and rehash entries.

Linear Probe Hashing



Hash Table – Key/Value Entries

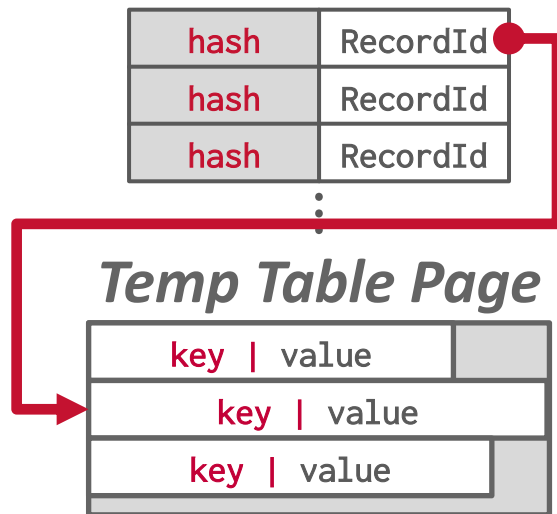
Fixed-length Key/Values:

- Store inline within the hash table pages.
- Optional: Store the key's hash with the key for faster comparisons.

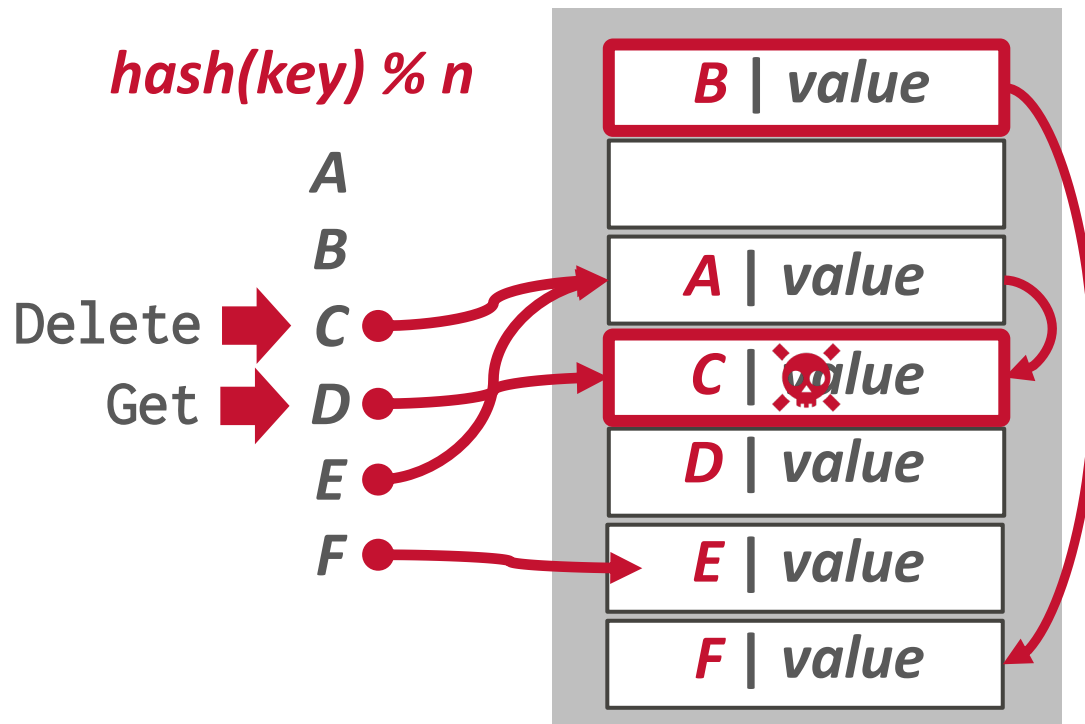
hash	key	value
hash	key	value
hash	key	value
⋮		

Variable-length Key/Values:

- Insert key/value data in separate a private temporary table.
- Store the hash as the key and use the record id pointing to its corresponding entry in the temporary table as the value.



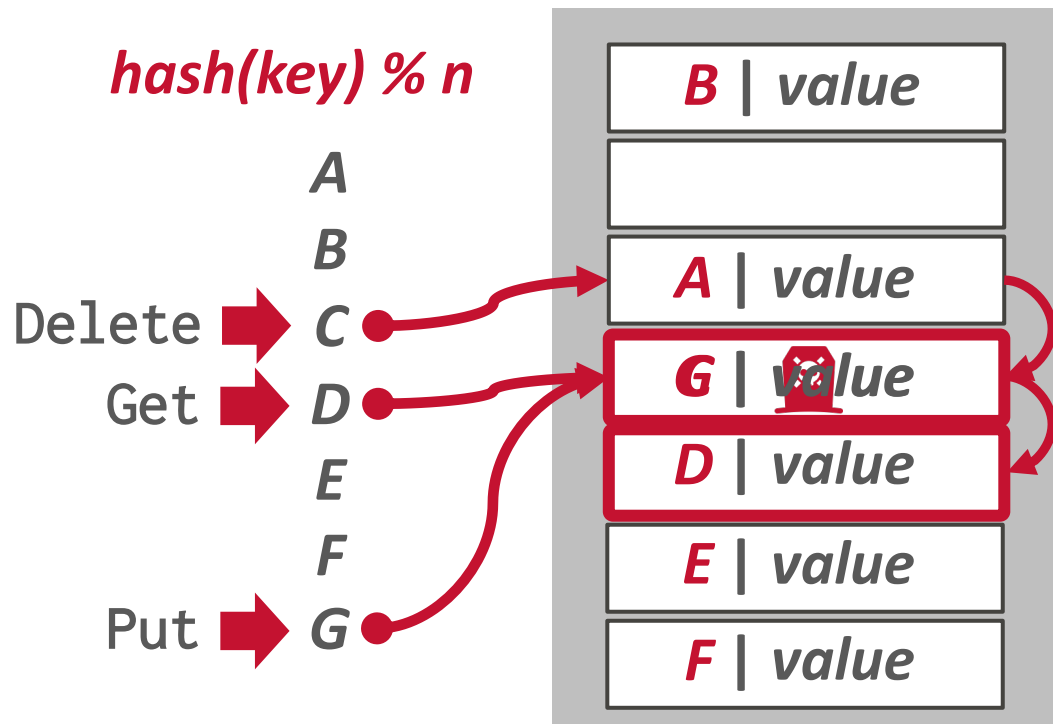
Linear Probe Hashing – Deletes



Approach #1: Movement

- Rehash keys until you find the first empty slot.
- Expensive! May need to reorganize the entire table.
- No DBMS does this.

Linear Probe Hashing – Deletes



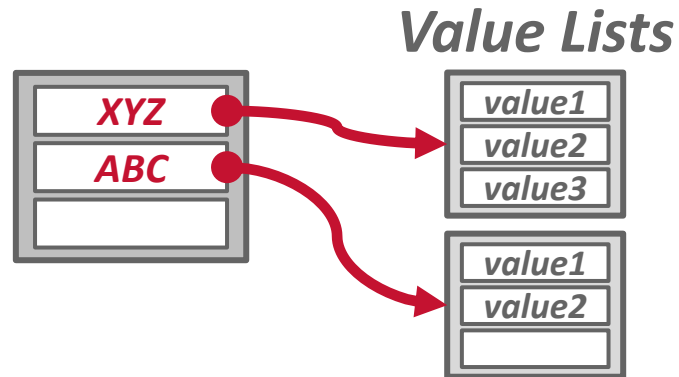
Approach #2: Tombstone

- Maintain separate bit map to indicate that the entry in the slot is logically deleted.
- Reuse the slot for new keys.
- May need periodic garbage collection.

Hash Table – Non-unique Keys

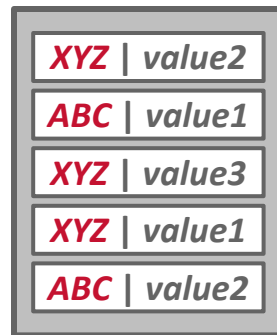
Choice #1: Separate Linked List

- Store values in separate storage area for each key.
- Value lists can overflow to multiple pages if the number of duplicates is large.



Choice #2: Redundant Keys

- Store duplicate keys entries together in the hash table.
- This is what most systems do.



Hash Table Latching

Easy to support concurrent access due to the limited ways threads access the data structure.

- All threads move in the same direction and only access a single page/slot at a time.
- Deadlocks are not possible.

To resize the table, take a global write latch on the entire table (e.g., in the header page).

Hash Table Latching

Approach #1: Page/Block Latches

- Each page/block has its own reader-writer latch that protects its entire contents.
- Threads acquire either a read or write latch before they access a page/block.

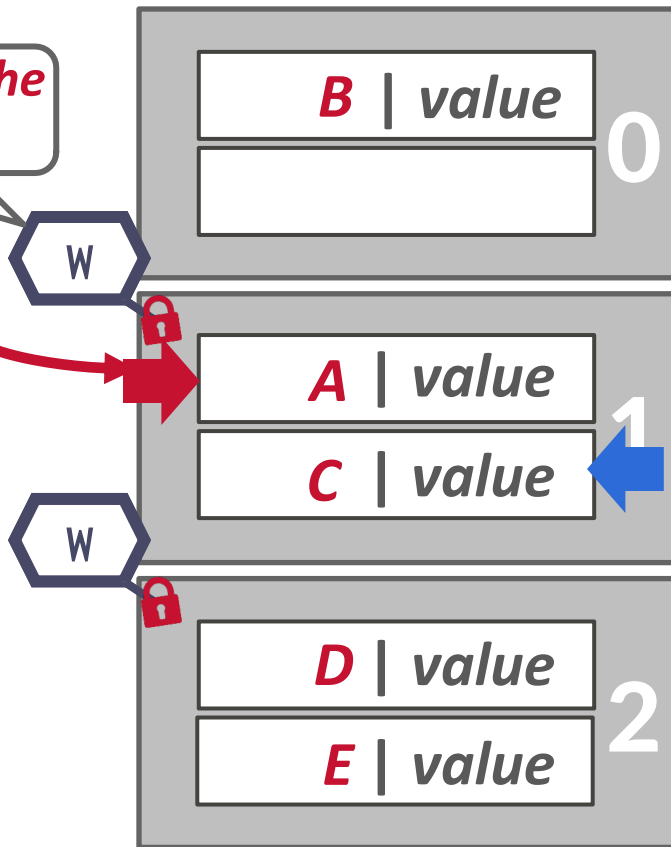
Approach #2: Slot Latches

- Each slot has its own latch.
- Can use a single-mode latch to reduce meta-data and computational overhead.

Hash Table: Page/Block Latches

It's safe to release the latch on Page #1.

T₁: Find D
hash(D)



T₂: Insert E

hash(E)

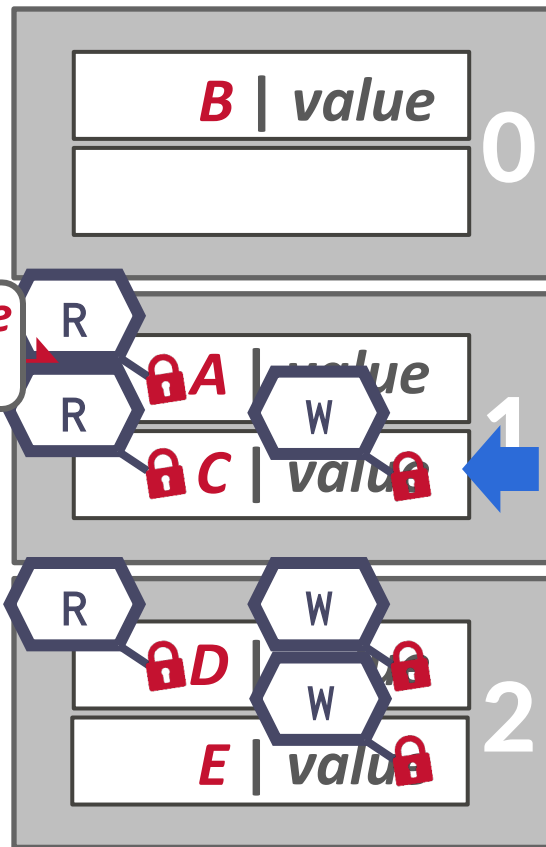


Hash Table: Slot Latches

T_1 : Find D

hash(D)

It's safe to release the latch on A



T_2 : Insert E
hash(E)

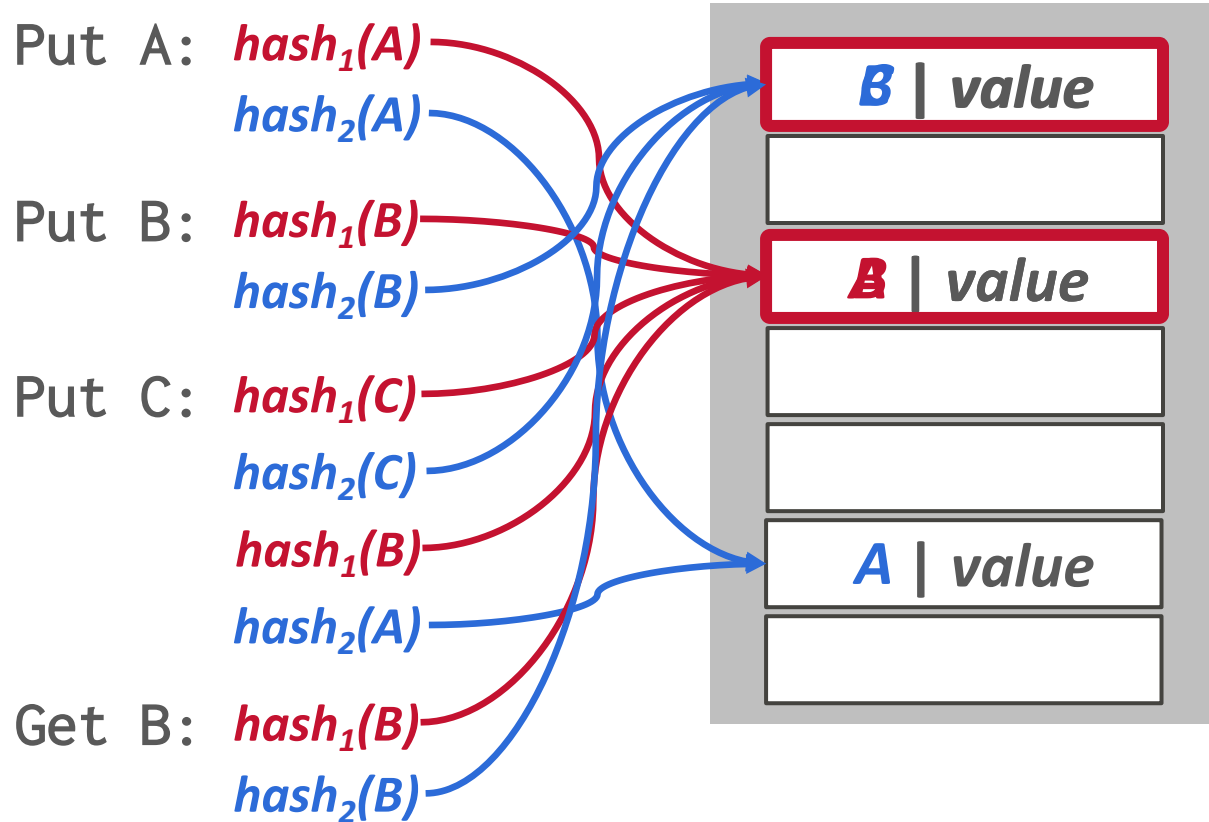
Cuckoo Hashing

Use multiple hash functions to find multiple locations in the hash table to insert records.

- On insert, check multiple locations and pick the one that is empty.
- If no location is available, evict the element from one of them and then re-hash it find a new location.

Look-ups and deletions are always **$O(1)$** because only one location per hash table is checked.

Cuckoo Hashing



Observation

The previous hash tables require the DBMS to know the number of elements it wants to store.

→ Otherwise, it must rebuild the table if it needs to grow/shrink in size.

Dynamic hash tables incrementally resize themselves as needed.

→ Chained Hashing

→ Extendible Hashing

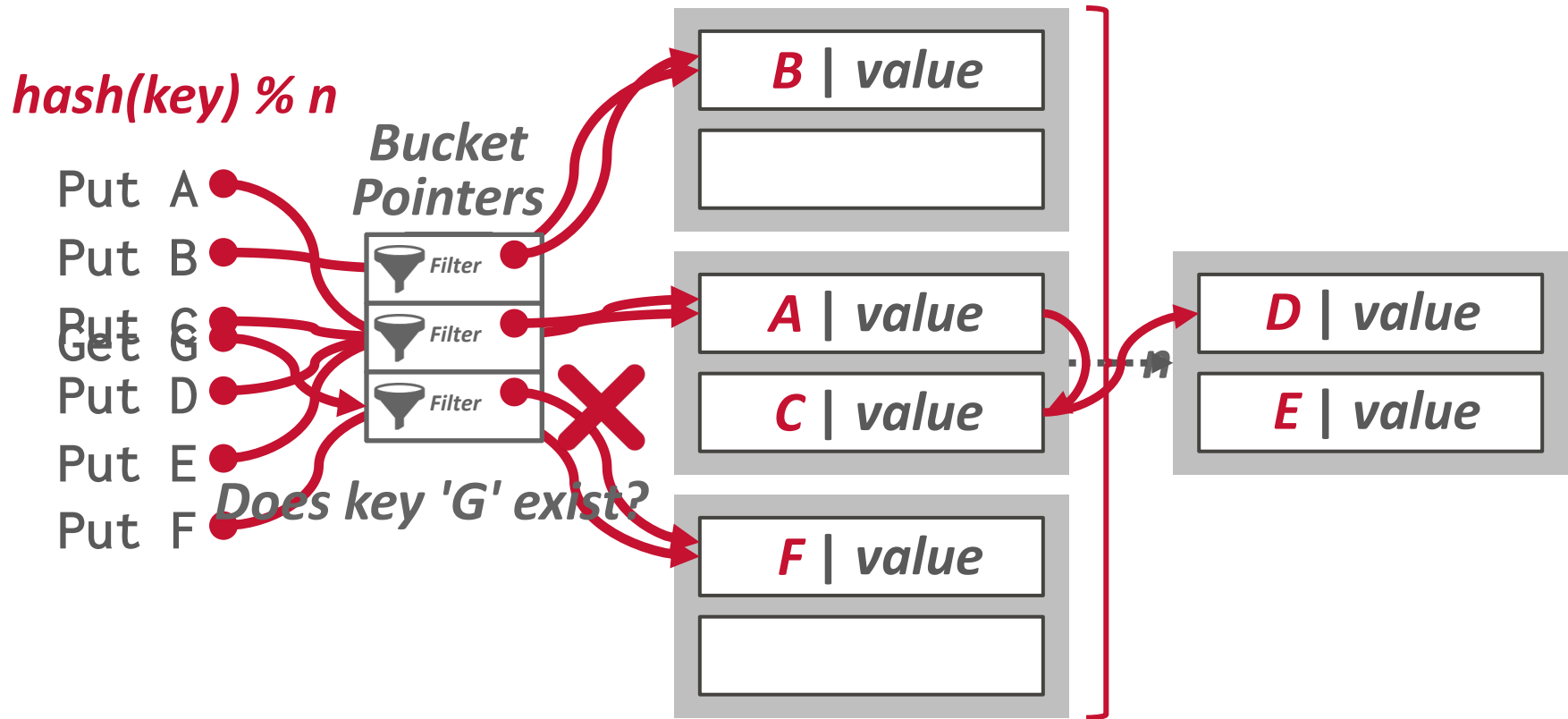
Chained Hashing

Maintain a linked list of buckets for each slot in the hash table.

Resolve collisions by placing all elements with the same hash key into the same bucket.

- To determine whether an element is present, hash to its bucket and scan for it.
- Insertions and deletions are generalizations of lookups.

Chained Hashing



Extendible Hashing

Chained-hashing approach that splits buckets incrementally instead of letting the linked list grow forever.

Multiple slot locations can point to the same bucket chain.

Reshuffle bucket entries on split and increase the number of bits to examine.

→ Data movement is localized to just the split chain.



Observation

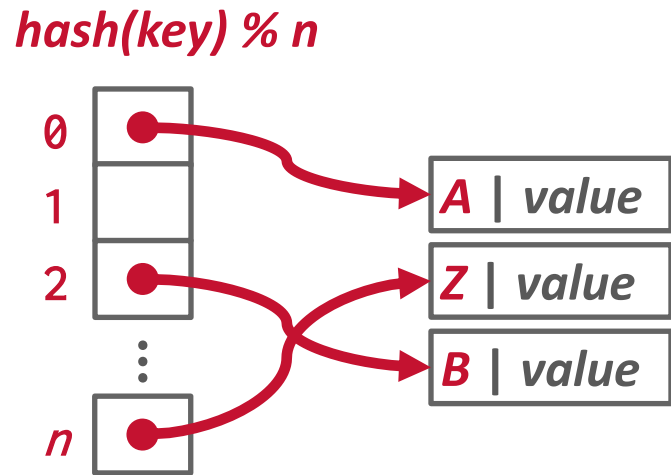
So far, n is fixed

- $\% n \approx$ "take the $\log n$ lowest bits"
- Let $i = \log n$

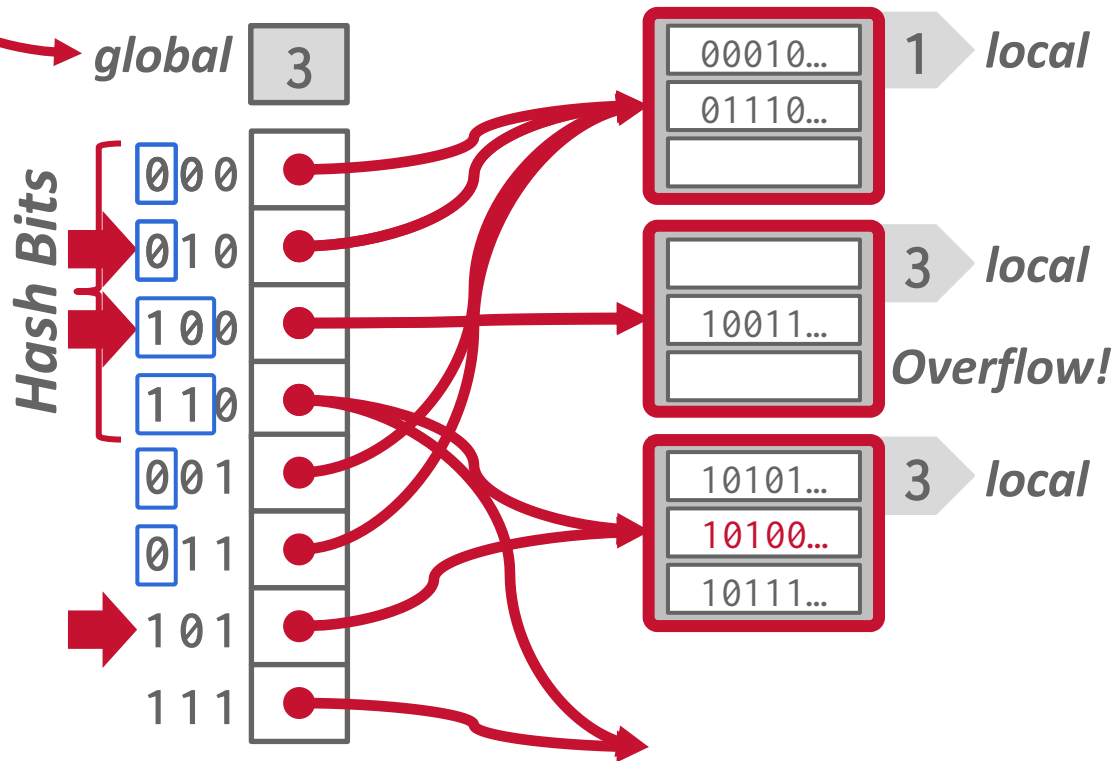
Mapping of i bits to bucket/slot is static

Some buckets/slots can fit all assigned keys (i is sufficiently large), some buckets/slots overflow (more bits needed)

Idea: Dynamically set i for each bucket to get the right number of keys



-Max number of bits to examine in hashes



Get A

$hash(A) = 01110...$

Put B

$hash(B) = 10111...$

Put C

$hash(C) = 10100\dots$

Conclusion

Fast data structures that support **$O(1)$** look-ups that are used all throughout DBMS internals.
→ Trade-off between speed and flexibility.

Hash tables are less common is table indexing because they are **unordered**

Next Class

Sorting...